Graphs, Self-Supervision and Deep Reinforcement Learning

Deep Learning course notes Sources: listed under References

What this file is really about:

$$\text{data with connections} \rightarrow \text{learning without labels} \rightarrow \text{policies trained directly}$$

How to use this file. Each part stands alone. Part B is complete code. Part C is practice with answers.

Step-up path

You need. Neural networks, attention and Q-learning.

1. Pass messages on a graph.
2. Add attention to a graph.
3. Learn representations without labels.
4. Derive the policy gradient.
5. Stabilise deep reinforcement learning.

You can stop when you can choose a method for graph, unlabelled or reward-driven problems.

Contents

Part A Maths From the Ground Up

A1 Why graphs

Molecules, social networks and road maps are not grids. A model must not depend on how the nodes happen to be numbered, so its layers must be **permutation equivariant**.

A2 Message passing

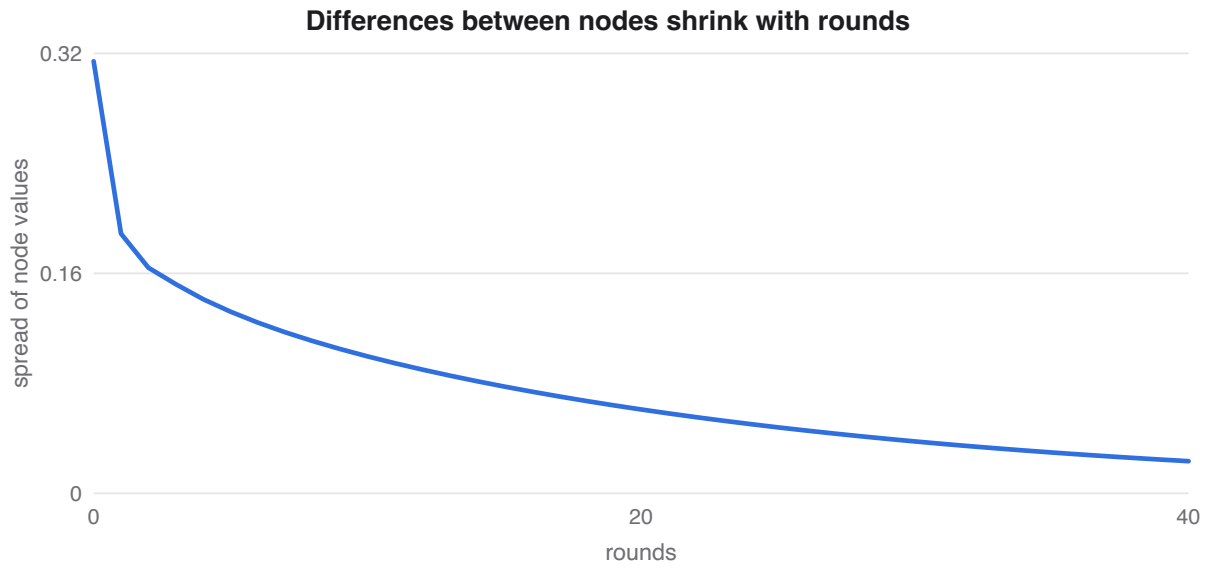
Each round, every node gathers its neighbours' states and updates its own:

$$h_v^{(k+1)} = \phi\Big(h_v^{(k)}, \bigoplus_{u \in N(v)} \psi(h_u^{(k)})\Big).$$

$\bigoplus$ must not depend on order: a sum, mean or maximum. A **graph convolution** in matrix form is

$$H' = \sigma\big(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}HW\big), \qquad \tilde{A} = A + I.$$

Careful. After many rounds, node states become alike. This **over-smoothing** limits depth.



A3 Graph attention

Rather than treating all neighbours equally, learn a weight for each:

$$\alpha_{vu} = \mathrm{softmax}_u\big(a(h_v, h_u)\big).$$

A transformer is graph attention on a fully connected graph.

A4 Self-supervised learning

Labels are expensive, and raw data are plentiful. Make up a task from the data itself.

- **Contrastive** methods such as SimCLR pull together two augmented views of one image, and push apart others.
- **Non-contrastive** methods such as BYOL and DINO use a slowly updated teacher, which prevents collapse without negatives.
- **Masked autoencoders** hide most of the image patches and reconstruct them.

Then evaluate the learned features using a **linear probe**: a linear classifier on frozen features.

A5 Policy gradients

Rather than learning values, directly improve a policy $\pi_\theta(a \mid s)$. The expected return has this gradient:

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\Big].$$

Why. Using $\nabla p = p \nabla \log p$, the gradient of an expectation becomes an expectation of a log-gradient, which we can estimate from samples. The dynamics do not depend on θ, so they vanish from the gradient.

Actions that led to high return become more likely. This is **REINFORCE**. Subtracting a baseline $b(s_t)$ does not change the expectation, and reduces variance.

A6 Actor-critic and PPO

An **actor-critic** learns a critic $V(s)$ and uses the advantage $A = r + \gamma V(s') - V(s)$ in place of G_t. **PPO** limits how far the policy moves in one update, by clipping the ratio of new to old probabilities:

$$L = \mathbb{E}\big[\min\big(rA,\ \text{clip}(r, 1-\epsilon, 1+\epsilon)A\big)\big].$$

A7 Deep Q-networks

A network estimates $Q(s, a)$ and regresses on $r + \gamma \max_{a'} Q_{\text{target}}(s', a')$. Replay memory and a slow target network keep this stable. **Double DQN** picks the action with the online network and evaluates it with the target network, which reduces overestimation.

Part B In Code

B1 A graph convolution layer

```python
import torch
from torch import nn


class GCN(nn.Module):
    def __init__(self, n_in, n_out):
        super().__init__()
        self.w = nn.Linear(n_in, n_out)

    def forward(self, h, adj):
        a = adj + torch.eye(len(adj))
        d = a.sum(1).pow(-0.5).diag()
        return torch.relu(self.w(d @ a @ d @ h))
```

B2 A contrastive loss

```python
import torch.nn.functional as F

def simclr_loss(z1, z2, tau=0.5):
    z = F.normalize(torch.cat([z1, z2]), dim=1)
    s = z @ z.T / tau
    n = len(z1)
    s.fill_diagonal_(float('-inf'))
    labels = torch.cat([torch.arange(n, 2 * n), torch.arange(n)])
    return F.cross_entropy(s, labels)
```

B3 REINFORCE

```
import torch
from torch import nn
from torch.distributions import Categorical

policy = nn.Sequential(nn.Linear(4, 64), nn.Tanh(), nn.Linear(64, 2))
opt = torch.optim.Adam(policy.parameters(), lr=1e-2)



def update(states, actions, rewards, gamma=0.99):
    G, returns = 0.0, []
    for r in reversed(rewards):
        G = r + gamma * G
        returns.insert(0, G)
    returns = torch.tensor(returns)
    returns = (returns - returns.mean()) / (returns.std() + 1e-8)
    dist = Categorical(logits=policy(torch.stack(states)))
    loss = -(dist.log_prob(torch.tensor(actions)) * returns).sum()
    opt.zero_grad(); loss.backward(); opt.step()
```

Part C Practice

P1. Why must a graph layer's aggregation not depend on order?

Node numbering is arbitrary, so renumbering must not change the result.

P2. After 3 rounds, how far can a node's information travel?

Three hops.

P3. Why does a baseline not bias the policy gradient?

$\mathbb{E}[\nabla \log \pi \cdot b(s)] = 0$, because $\sum_a \nabla \pi(a \mid s) = \nabla 1 = 0$.

P4. What is a linear probe?

A linear classifier trained on frozen features, used to measure how good a representation is.